

Analizzatore statico basato sull'interpretazione astratta

Università degli studi di Verona - Dipartimento di
Informatica

Alessio Terraroli

Anno accademico 2025/2026

Indice

1	Nozioni logico-matematiche	2
1.1	Domini Astratti, Poset e Reticoli	2
1.2	L'operatore Least Upper Bound (LUB, $\sqcup$)	2
1.3	Calcolo del Punto Fisso (Fixpoint Computation) e Widening	2
1.4	Conservatività e Aggiornamenti Deboli (Weak Updates)	3
2	Introduzione teorica	4
2.1	Analisi statica	4
2.2	Interpretazione Astratta	4
3	Introduzione	6
4	Background Teorico	6
4.1	Interpretazione Astratta	6
4.2	Soundness e Approssimazione	6
4.3	LUB e Convergenza (Fixpoint)	6
5	Il Dominio dei Segni (Sign Lattice)	6
5.1	Struttura del Reticolo Esteso	7
5.2	Operazioni Aritmetiche Astratte	7

1 Nozioni logico-matematiche

1.1 Domini Astratti, Poset e Reticoli

L'interpretazione astratta sostituisce il dominio infinito dei valori concreti di esecuzione di un programma con un dominio matematico finito e semplificato, noto come **Dominio Astratto**[cite: 2]. Questo dominio è formalmente strutturato come un *Poset* (Partially Ordered Set, o insieme parzialmente ordinato) o un reticolo completo (*Lattice*)[cite: 2]. Gli estremi di questo reticolo sono identificati da due simboli fondamentali:

- $\perp$ (**Bottom**): Rappresenta il limite inferiore del reticolo. Indica uno stato non inizializzato, del codice irraggiungibile o un errore statico a runtime (ad esempio, una divisione per zero)[cite: 1, 2]. Nel contesto del dominio delle stringhe modellato tramite Automi a Stati Finiti (FSA), il Bottom corrisponde a un automa vuoto[cite: 1].
- $\top$ (**Top**): Rappresenta il limite superiore del reticolo, ovvero l'incertezza totale[cite: 1, 2]. Questo stato viene innescato quando l'esecuzione su percorsi sovrapposti produce segni discordanti o genera ambiguità matematica[cite: 1, 2]. Nel dominio delle stringhe, Top rappresenta la stringa universale, ovvero un automa che accetta qualsiasi stringa[cite: 1].

1.2 L'operatore Least Upper Bound (LUB, $\sqcup$)

Il **Least Upper Bound**, denotato con il simbolo $\sqcup$, è l'operatore matematico utilizzato per calcolare il limite superiore meno stringente tra due valori all'interno del reticolo[cite: 1, 2]. Il suo ruolo è essenziale durante la valutazione delle diramazioni condizionali (come i costrutti If-Else): poiché il valore di guardia potrebbe essere valutato come $\top$, l'interprete esplora entrambi i percorsi di esecuzione in modo speculativo[cite: 1, 2]. Al punto di giunzione, le memorie astratte finali derivanti dai due rami vengono riconciliate fondendole tramite il LUB:

$$\text{Memory}_{\text{final}} = \text{Memory}_{\text{Then}} \sqcup \text{Memory}_{\text{Else}}$$

[cite: 2] Nel dominio astratto degli automi a stati finiti per l'analisi delle stringhe, l'applicazione dell'operatore LUB corrisponde direttamente al calcolo dell'unione matematica dei due automi[cite: 1].

1.3 Calcolo del Punto Fisso (Fixpoint Computation) e Widening

La risoluzione dei cicli (*While*) è uno dei problemi centrali dell'analisi statica. Per aggirare il problema della fermata (*Halting Problem*), l'interprete esegue una **Fixpoint Computation** (calcolo del punto fisso)[cite: 1, 2]. Il corpo del ciclo viene valutato iterativamente unendo il nuovo stato con quello precedente tramite l'operatore LUB:

$$M_{i+1} = M_i \sqcup \text{body}(M_i)$$

[cite: 1, 2] L'iterazione termina quando la memoria astratta raggiunge uno stato stazionario (il punto fisso), momento in cui ulteriori iterazioni non produrrebbero alcun cambiamento[cite: 1, 2]. Per domini con un'altezza finita, come il Reticolo

dei Segni, la convergenza (spesso verso $\top$) è matematicamente garantita in pochi millisecondi[cite: 1, 2].

Quando si utilizzano domini potenzialmente infiniti, come gli automi per le stringhe o le astrazioni per gli array, un ciclo potrebbe far crescere il valore astratto all'infinito[cite: 1]. In questi casi, il calcolo del punto fisso introduce un operatore di **Widening**[cite: 1]. Il widening forza una perdita volontaria di precisione (ad esempio, degradando l'automa a $\top$ se il numero dei suoi stati supera una certa soglia), garantendo la terminazione rapida e sicura dell'analisi[cite: 1].

1.4 Conservatività e Aggiornamenti Deboli (Weak Updates)

L'analizzatore statico deve garantire un'approssimazione sicura (*Sound Approximation*): non deve mai assumere proprietà false, pur accettando una perdita di precisione (conservatività)[cite: 1, 2]. Questo principio è cruciale nella gestione di strutture dati mutabili e complesse, come l'astrazione degli Array in modalità *Index-Sensitive*[cite: 1]. Quando l'indice a cui si tenta di accedere non è noto staticamente (viene valutato come $\top$), l'interprete non sa con certezza quale cella della memoria andrà a modificare[cite: 1]. Invece di sovrascrivere un valore rischiando di compiere un'operazione errata (*strong update*), l'analizzatore è costretto ad applicare un **Weak Update** (aggiornamento debole)[cite: 1]. L'aggiornamento debole fonde (tramite l'operatore LUB, $\sqcup$) i nuovi dati in ingresso con i dati astratti già presenti, assicurando una stima conservativa che preserva la validità dell'analisi senza sovrascrivere indebitamente stati validi[cite: 1].

2 Introduzione teorica

2.1 Analisi statica

L'analisi statica è una tecnica utilizzata per esaminare il codice sorgente di un programma senza eseguirlo. Questa tecnica consente di identificare potenziali errori, vulnerabilità di sicurezza, e problemi di stile nel codice. L'analisi statica può essere eseguita manualmente da sviluppatori esperti o automaticamente utilizzando strumenti software dedicati. Gli strumenti di analisi statica possono essere configurati per rilevare una vasta gamma di problemi, tra cui errori di sintassi, problemi di sicurezza, e violazioni delle convenzioni di codifica. L'analisi statica è una parte importante del processo di sviluppo del software, poiché aiuta a migliorare la qualità del codice e a ridurre il numero di bug e vulnerabilità che possono essere introdotti durante lo sviluppo.

Un interprete è un programma che esegue direttamente le istruzioni scritte in un linguaggio di programmazione, senza la necessità di compilare il codice in un formato eseguibile. L'interprete legge il codice sorgente, lo analizza e lo esegue riga per riga o blocco per blocco. Questo processo consente agli sviluppatori di testare e modificare rapidamente il codice, poiché non è necessario attendere la compilazione completa del programma. Gli interpreti sono spesso utilizzati durante la fase di sviluppo e debugging, poiché offrono un feedback immediato sugli errori e sui risultati dell'esecuzione del codice.

Sebbene l'esecuzione concreta (o l'interpretazione classica) fornisca risultati esatti per una specifica esecuzione, non permette di dedurre proprietà generali del programma per tutti i possibili input. D'altra parte, il Teorema di Rice stabilisce che verificare proprietà semantiche non banali di un programma (come la terminazione o l'assenza di errori a runtime) è un problema indecidibile. Per cercare di affrontare questa limitazione, l'analisi statica moderna si affida a tecniche di approssimazione matematica.

2.2 Interpretazione Astratta

L'Interpretazione astratta, introdotta da Patrick e Radhia Cousot nel 1977, fornisce una teoria per l'approssimazione sicura (*sound*) delle strutture matematiche, applicata alla semantica formale dei programmi. Poiché non è possibile calcolare esattamente tutti gli stati infiniti di un programma, un interprete astratto sostituisce il dominio infinito dei valori concreti di esecuzione con un dominio semplificato e finito, chiamato **dominio astratto**. Questo dominio è tipicamente strutturato matematicamente come un *poset* (Partially Ordered Set) o un reticolo (*Lattice*).

Il principio fondamentale di questa tecnica è la *conservativeness* (affidabilità) in cui l'analizzatore calcola uno stato approssimato ma matematicamente sicuro. L'interprete garantisce di non assumere mai proprietà false, pur accettando una perdita di precisione per mantenere l'analisi decidibile. Se durante l'analisi un'operazione produce incertezza o ambiguità matematica, il sistema degrada verso uno stato di totale ignoranza, comunemente rappresentato dal simbolo $\top$ (Top), evitando così di compiere deduzioni non sicure (*unsound*). Al contrario, lo stato di errore, codice

irraggiungibile o variabile non inizializzata è rappresentato dal simbolo $\perp$ (Bottom).

L'astrazione si riflette anche nella valutazione del flusso di controllo. Quando l'interprete incontra diramazioni condizionali (If-Else), la guardia statica potrebbe valutarsi come $\top$ (incerta). In tali scenari, l'interprete esplora speculativamente entrambi i percorsi di esecuzione. Le memorie astratte risultanti dai due rami vengono poi riconciliate al punto di giunzione (*join-point*) avvalendosi dell'operatore matematico **Least Upper Bound** (LUB, $\sqcup$). Questo operatore calcola il minimo limite superiore nel reticolo, unificando gli stati discordanti.

La sfida maggiore nell'analisi statica è la valutazione dei cicli (While), poiché tentare di srotolarli all'infinito porterebbe all'Halting Problem. L'astrazione ridefinisce la dimensione dei possibili stati tramite l'utilizzo di domini astratti, ovvero domini finiti che forniscono una rappresentazione approssimata ma sicura degli stati del programma. I domini astratti, diversamente dai domini matematici concreti, invece di essere definiti da un insieme di valori univoci o di intervalli, sono definiti da proprietà dei valori stessi, ad esempio la parità o la disparità di un valore intero oppure, come mostrato all'interno di questo progetto, il segno.

L'interprete valuta iterativamente il corpo del ciclo, unendo continuamente lo stato della memoria successiva con quello precedente tramite l'operatore LUB ($M_{i+1} = M_i \sqcup \text{body}(M_i)$). Il calcolo termina quando l'iterazione non porta più ad alcun cambiamento, ovvero quando la memoria astratta raggiunge uno stato stazionario (il punto fisso). Poiché il dominio astratto (come il reticolo dei segni) ha un'altezza finita, la convergenza è matematicamente garantita, permettendo all'interprete di analizzare loop potenzialmente infiniti in pochi millisecondi.

3 Introduzione

Il progetto consiste in un interprete astratto sviluppato in Java, progettato per eseguire l'analisi statica di un linguaggio di programmazione. L'architettura si basa sull'uso di **ANTLR4** per l'analisi sintattica, operando direttamente sull'albero sintattico (Parse Tree) tramite il pattern *Visitor*, bypassando rappresentazioni intermedie complesse come AST o CFG. Il framework implementa un *Type System* statico e calcola la semantica astratta per verificare in modo sicuro le proprietà del programma senza eseguirlo.

4 Background Teorico

4.1 Interpretazione Astratta

Introdotta da Patrick e Radhia Cousot nel 1977, l'Interpretazione Astratta fornisce una teoria per l'approssimazione sicura (*sound approximation*) delle strutture matematiche, applicata principalmente alla semantica formale dei programmi. Poiché verificare proprietà semantiche non banali è indecidibile (Teorema di Rice), un interprete astratto sostituisce il dominio infinito dei valori concreti di esecuzione con un **Dominio Astratto** finito, strutturato come reticolo (*Lattice*).

4.2 Soundness e Approssimazione

L'analizzatore calcola uno stato approssimato ma sicuro: garantisce di non assumere mai proprietà false, pur potendo perdere precisione per mantenere la decidibilità. In presenza di incertezza, il sistema degrada verso uno stato di totale ignoranza ($\top$).

4.3 LUB e Convergenza (Fixpoint)

- **Diramazioni Condizionali (If-Else):** Poiché la guardia statica potrebbe valutarsi come $\top$, l'interprete esplora speculativamente entrambi i rami. Le memorie astratte finali vengono riconciliate al *join-point* usando l'operatore matematico **Least Upper Bound (LUB, $\sqcup$)**:

$$\text{Memory}_{\text{final}} = \text{Memory}_{\text{Then}} \sqcup \text{Memory}_{\text{Else}}$$

- **Cicli (While):** Per risolvere staticamente il problema della fermata (*Halting Problem*), l'analisi dei loop esegue una **Fixpoint Computation** (calcolo del punto fisso). L'interprete valuta iterativamente il corpo del ciclo finché la memoria astratta non raggiunge uno stato stazionario:

$$M_{i+1} = M_i \sqcup \text{body}(M_i)$$

Grazie all'altezza finita del reticolo dei segni, la convergenza in $\top$ è matematicamente garantita in pochi millisecondi senza il rischio di loop infiniti.

5 Il Dominio dei Segni (Sign Lattice)

Il dominio concreto degli interi $\mathbb{Z}$ viene astratto nel Dominio dei Segni. Nel progetto, il reticolo base è stato esteso per supportare un'analisi più granulare e precisa, in preparazione all'analisi dei linguaggi dinamici.

5.1 Struttura del Reticolo Esteso

Il reticolo attuale, definito dalla classe `SignType`, supporta le seguenti astrazioni:

- $\perp$ (**Bottom**): Rappresenta uno stato non inizializzato, codice irraggiungibile o un errore statico a runtime (es. divisione per zero).
- **NEG** ($-$): Numeri strettamente negativi.
- **ZERO** (0): Esattamente zero.
- **POS** ($+$): Numeri strettamente positivi.
- **ZERO_PLUS** ($0+$): Numeri maggiori o uguali a zero (zero o positivi).
- **ZERO_MINUS** ($0-$): Numeri minori o uguali a zero (zero o negativi).
- **NOT_ZERO** ($\neq 0$): Numeri positivi o negativi, ma certamente non zero. Generato ad esempio dal LUB tra **POS** e **NEG**.
- $\top$ (**Top**): Incertezza totale (potrebbe essere qualsiasi intero).

5.2 Operazioni Aritmetiche Astratte

La classe `SignValue` è responsabile della gestione delle operazioni aritmetiche tra gli elementi del reticolo. Tali operazioni garantiscono l'affidabilità (*soundness*) dell'analisi:

- **Addizione** (`add`): Mantiene la precisione se i segni concordano (es. **POS** + **POS** = **POS**, **POS** + **0+** = **POS**). Se si sommano valori opposti o misti che coprono l'intero asse numerico (es. **POS** + **NEG** oppure **NOT_ZERO** + **0+**), il risultato degrada in $\top$.
- **Sottrazione** (`sub`): L'operatore meno unario inverte il segno (es. **POS** $\rightarrow$ **NEG**), permettendo di calcolare la sottrazione tramite l'addizione con l'operando invertito.
- **Moltiplicazione** (`mul`): Calcola accuratamente i prodotti combinati, restituendo valori come **0+** o **0-** quando si moltiplicano segni incerti (che includono lo zero) con segni certi.
- **Divisione** (`div`): Gestisce in modo conservativo il troncamento della divisione intera in Java. Ad esempio, la divisione tra **POS** e **POS** restituisce **0+**, considerando il caso in cui il numeratore sia minore del denominatore (es. $1/2 = 0$). Se vi è certezza di dividere per zero, viene restituito lo stato di errore $\perp$.